

Kincsvadászat

Időlimit: 8 s Memórialimit: 256 MiB

Rég elveszett kincsek után kutatsz egy hatalmas, $N \times N$ cellából álló rácson a varázsiránytűd segítségével. Összesen $K \leq 3$ kincsesláda van elrejtve a mező K különböző cellájában. A célod egyszerű: találd meg mindegyiket!

Minden lépésben egy általad választott cellára helyezheted az iránytűt. Amikor az iránytűt egy cellára helyezed, az megkeresi az adott cellából induló **összes** létező legrövidebb utat, ami kincsesládához vezet, feltéve, hogy csak balra, felfelé, jobbra és lefelé lehet mozogni. Ezután megmutatja az **összes** lehetséges első lépés irányát. Vedd figyelembe, hogy ez jelentheti azt, hogy egyetlen legközelebbi kincsesláda van (amihez egy vagy több lehetséges első lépés vezethet el), de akár jelentheti azt is, hogy több különböző kincsesláda van azonos legkisebb távolságra (és ekkor mindegyik ilyen kincsesládához vezető összes legrövidebb út első lépéseit megkapod). Ha a cella maga tartalmaz kincset, akkor az iránytű ezt a tényt jelzi az irányok helyett.

Valahányszor megtalálsz egy kincsesládát, megszerzed a tartalmát, de magát a ládát nem tudod eltávolítani, mert túl nehéz. Az iránytű nem tudja, hogy egy láda tele van-e vagy üres: mindig a legközelebbi láda felé mutat, függetlenül attól, hogy üres vagy tele van.

Keresd meg az összes kincsesládát az iránytű lehető legkevesebb használatával.

Könyvtár

Ez egy interaktív feladat. Minden tesztelésben (vagyis a programod minden futtatásakor) a programodnak egy vagy több kincsvadászatot kell megoldania. Az értékelő programmal függvényhívások segítségével tudsz kommunikálni. A rendelkezésedre álló könyvtár a következő deklarációkat tartalmazza:

- **void** *NextHunt*(**int** &*N*, **int** &*K*) — hívd meg ezt a függvényt a soron következő kincsvadászat elindításához. Ez beállítja a rács méretét az általad átadott *N* változóban, valamint a keresett kincsek számát a *K* változóban. Ha az aktuális futás során már nincs több megoldandó kincsvadászat, a függvény *N*-et és *K*-t -1 -re állítja; ebben az esetben a programodnak 0 kilépési kóddal be kell fejeznie a futását. **Fontos**, hogy ezt a függvényt azelőtt is meghívhatod, hogy az aktuális vadászat összes kincsét megtaláltad volna, például ha csak részpontszámot szeretnél elérni.
- **enum** { *TREASURE* = 0, *DIR_RIGHT* = 1, *DIR_UP* = 2, *DIR_LEFT* = 4, *DIR_DOWN* = 8 }; — ezek azok a konstansok, amelyeket a *Query* függvény viszatérési értékei használnak (lásd lentebb).
- **int** *Query*(**int** *x*, **int** *y*) — ha az (x, y) koordinátájú cella kincset tartalmaz, akkor a függvény *TREASURE*-t ad vissza. Ellenkező esetben visszaadja a *DIR_RIGHT*, *DIR_UP*, *DIR_LEFT* és *DIR_DOWN* konstansok közül azoknak az **összegét**, amely irányú mozgásokat az iránytű megad, amikor az (x, y) cellára helyezed. Az *x* és *y* koordinátáknak 0 és $N - 1$ közötti egész számoknak kell lenniük. **Az *y* koordináták fentről lefelé növekednek.**

Miután a *NextHunt* beállította N és K értékét -1 -re, a programod többé nem hívhatja meg sem a *NextHunt*, sem a *Query* függvényt. A *Query* függvényt az első *NextHunt* hívás előtt sem szabad meghívni. Ha a programod nem tartja be ezeket a feltételeket, vagy egyetlen kincsvadászaton belül több mint 1000 lekérdezést hajt végre, vagy a *Query* hívásakor az x és/vagy y koordináták a megengedett tartományon kívül esnek, akkor a könyvtár leállítja a programot, és futási hiba (RTE) eredményt ad az aktuális tesztesetre.

Akkor tekintjük úgy, hogy egy kincset megtaláltál, ha legalább egyszer lekérdezted azt a cellát, amelyben található. Ha a programod meghívja a *NextHunt* függvényt, mielőtt az aktuális kincsvadászat összes kincsét megtalálta volna, az nem számít hibának, de befolyásolja a pontszámot (erről lentebb részletesebben).

A könyvtár használatához a programodnak tartalmaznia kell a

```
#include "treasurehuntlib.h"
```

importáló sort. Az importálandó fájl innen tölthető le: `treasurehuntlib.h`.

A megoldás elkészítésének segítésére a könyvtár egy lokálisan használható implementációja is rendelkezésre áll itt: `treasurehuntlib-public.cpp`. Ahhoz, hogy ezt a programoddal együtt lefordítsd, add hozzá a nevét a fordító paramétereinek, például így:

```
g++ foo.cpp treasurehuntlib-public.cpp
```

ahol a `foo.cpp` a megoldásodat tartalmazó fájl neve.

A lokális teszteléshez letölthető `treasurehuntlib-public.cpp` a korábban felsoroltakon túl deklarálja a `void InitFromFile(const char *fileName)` függvényt is. Ez egy vagy több kincsvadászat leírását olvassa be a paraméterként megadott nevű fájlból. Ezt a programod elején kell meghívnod és lehetővé teszi, hogy a *NextHunt* hívások során a fájlban található kincsvadászatokat játszd le, a véletlenszerűen generáltak helyett. További leírást a használatáról (például a bemeneti fájl formátumáról) a `treasurehuntlib-public.cpp` fájlban találsz.

A kiértékelő szerveren a könyvtár egy másik implementációját fogják használni, ezért nem szabad semmilyen feltételezést tenned annak pontos működéséről. Feltételezheted azonban, hogy a globális névtér nem tartalmaz az itt felsoroltakon (*NextHunt*, *Query* és az öt konstans) kívül további deklarációkat.

A programod nem olvashat a standard bemenetről és nem írhat a standard kimenetre, mivel ezeket a kiértékelő szerveren futó könyvtárimplementációnk az értékelőrendszer többi részével való kommunikációra használja.

Bemenet

Korlátok

- $2 \leq N \leq 1\,000\,000$
- $1 \leq K \leq 3$
- Egyetlen programfuttatás során legfeljebb 100 000 kincsvadászat fordul elő.
- Egyetlen kincsvadászaton belül legfeljebb 1000 lekérdezés hajtható végre.
- Az értékelő **nem adaptív**, ami azt jelenti, hogy a rácsok mérete és a kincsek pozíciói a programfuttatás előtt rögzítve lettek.

Részfeladatok

1. részfeladat (10 pont): $K = 1$.
2. részfeladat (30 pont): $K = 2$.
3. részfeladat (60 pont): $K = 3$.

Pontozás

Egy részfeladat több tesztesetből (a program több futtatásából) állhat, és minden teszteset több kincsvadászatot tartalmazhat. A pontozás szempontjából egy adott részfeladat összes kincsvadászatát együtt értékelik, függetlenül attól, hogy eredetileg hogyan oszlottak meg a tesztesetek között. Az i -edik kincsvadászat esetén jelölje a rács méretét N_i , a kincsek számát K_i , a programod által végrehajtott lekérdezések számát Q_i , a megtalált kincsek számát pedig F_i . Továbbá jelölje S az adott részfeladatra szerezhető összes pontot. Ekkor a programod által elért pontszám a részfeladatban:

- Ha a programod minden kincsvadászatban megtalálta az összes kincset (azaz $F_i = K_i$ minden i -re), akkor a pontszáma a $t_i = \frac{Q_i}{\lceil \log_2 N_i \rceil}$ értéktől függ:

$$\frac{S}{2} + \frac{S}{2} \cdot \min_i f(t_i) \text{ pont, ahol } f(t_i) = \begin{cases} 1, & t_i \leq 11 \\ 1 - (t_i - 11)/9, & 11 \leq t_i \leq 20 \\ 0, & t \geq 20. \end{cases}$$

- Ha a programod nem minden esetben találta meg az összes kincset (azaz $F_i < K_i$ legalább egy i -re), akkor a pontszáma:

$$\frac{S}{2} \cdot \min_i \frac{F_i}{K_i} \text{ pont.}$$

Más szóval, a pontok felét az összes kincs megtalálásáért kapod, a másik felét pedig azért, hogy ezt minél kevesebb lekérdezéssel teljesítsd.

- A maximális pontszámhoz a megoldásodnak legfeljebb $11 \lceil \log_2 N_i \rceil$ lekérdezéssel kell megtalálnia az összes kincset.
- $11 \lceil \log_2 N_i \rceil$ és $20 \lceil \log_2 N_i \rceil$ lekérdezés között a pontszám lineárisan csökken;
- ha pedig a megoldásod több, mint $20 \lceil \log_2 N_i \rceil$ lekérdezést használ, akkor csak az összes kincs megtalálásáért járó pontokat kapja meg. (A $\lceil \cdot \rceil$ jelölés azt jelenti, hogy a $\log_2 N_i$ értékét a legközelebbi nagyobb, vagy egyenlő egész számra kerekítjük.)

<!--If the formulas above result in a non-integer number of points for the subtask, it will be rounded to the nearest integer.

If your program has a runtime error or doesn't adhere to the aforementioned protocol in using the library, it will get 0 points for the whole subtask. To receive partial points for finding a subset of the treasures, your program therefore needs to finish the search gracefully by calling *NextHunt*.-->

Ha a fenti képletek nem egész pontszámot eredményeznek egy részfeladatra, akkor azt a legközelebbi egész számra kerekítik.

Ha a programod futási hibát okoz, vagy nem tartja be a könyvtár használatának fent ismertetett protokollját, akkor 0 pontot kap az egész részfeladatra. Ahhoz, hogy részpontszámot kapj a kincsek csak egy részhalmazának megtalálásáért, a keresést szabályosan kell befejezned a *NextHunt* meghívásával.

Példa

Függvényhívás	Visszkapott értékek
<i>NextHunt</i> (N, K)	$N = 4, K = 1$
<i>Query</i> (2, 0)	$9 = DIR_DOWN + DIR_RIGHT$
<i>Query</i> (3, 1)	$8 = DIR_DOWN$
<i>Query</i> (3, 2)	$0 = TREASURE$
<i>NextHunt</i> (N, K)	$N = -1, K = -1$